

Compiler Design

Chapter Four

Semantic Analysis

Contents

2

1. Syntax Directed Translation
2. Syntax Directed Definitions
3. Construction of syntax trees
4. L-attributed definitions and S-Attribute
5. Bottom-up evaluation of S-attributed definitions

Introduction

3

- Semantic analysis is the **third phase of compiler**.
- Semantic analysis make sure that **declaration and statement of the program** are semantically correct.
 -
- Gives meaning for **each and every elements** such as **tokens and syntax structure** etc.
 -
- Check datatypes have **proper meaning or not**.
- Check the **syntax structure** of the source code
- Check meanings of the **labels, controls, loops and switches**.

Need of semantic analysis

4

- Give assurance of the declaration of the statements of program are correct.
- Proper **datatypes** are used or not.
- Check whether variables are **declared before used** in the program.
- Check **datatypes, constraints, expression and assignment** in the program.
- Check **number of parameters** used in the function calls.

Way of representation

5

- **There are two ways to represent the semantic rules associated with grammar symbols.**
-
- **Syntax-Directed Translation Schemes (SDT)**
- **Syntax-Directed Definitions (SDD)**

Syntax Directed Definition (SDD)

6

- **Check value of the attributes by related to semantic rules of the grammar.**
- Don't specify order of evaluation of the semantic action.
- Easily read and write.
- Types of attributes:
 - S - attributes
 - L – attributes

Syntax Directed Translation (SDT)

7

- Perform semantic action with in procedure bodies.
- **Construction where translation of the source language is done by parser.**
- **More efficient and easy to implement.**
- **Types of attribute:**
 1. **L – attribute**
 2. **S – attribute**

Syntax Directed Definition (SDD) cont....

9

- Syntax Directed Definitions are a generalization of context-free grammars in which
- Grammar symbols have an associated set of Attributes;
- Productions are associated with Semantic Rules for computing the values of attributes.
- Such formalism generates Annotated Parse-Trees where each node of the tree is a record with a field for each attribute (e.g., X.a indicates the attribute a of the grammar symbol X).
- The value of an attribute of a grammar symbol at a given parse-tree node is defined by a semantic rule associated with the production used at that node.

Syntax Directed Definition (SDD) Attribute Grammar

9

- Attributes are properties associated with grammar symbols. Attributes can be numbers, strings, memory locations, data types, etc.
- Attribute grammar is a special form of context-free grammar where some additional information (attributes) are appended to one or more of its non-terminals in order to provide context-sensitive information.
- Attribute grammar is a medium to provide semantics to the context-free grammar and it can help specify the syntax and semantics of a programming language.

Attribute Grammar

9

- Attribute Grammar
- Example: $E \rightarrow E + T \{ E.value = E.value + T.value \}$
- The right part of the CFG contains the semantic rules that specify how the grammar should be interpreted. Here, the values of non-terminals E and T are added together and the result is copied to the non-terminal E.
- Semantic attributes may be assigned to their values from their domain at the time of parsing and evaluated at the time of assignment or conditions.
- Based on the way the attributes get their values, they can be broadly **divided into two categories : synthesized attributes and inherited attributes**

SDD

9

- **S - Attribute (synthesized attribute):** these are those nodes which drive their values from there children nodes.

```

S  --> E
E  --> E1 +
T  --> T
T  --> T1 *
F  --> F
F  --> digit
    
```

- Step 1: write SDD using appropriate semantic rules for each production.

PRODUCTIONS	SEMANTIC RULES
$S \rightarrow E$	$\text{Print}(E.val)$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

SDD cont..

9

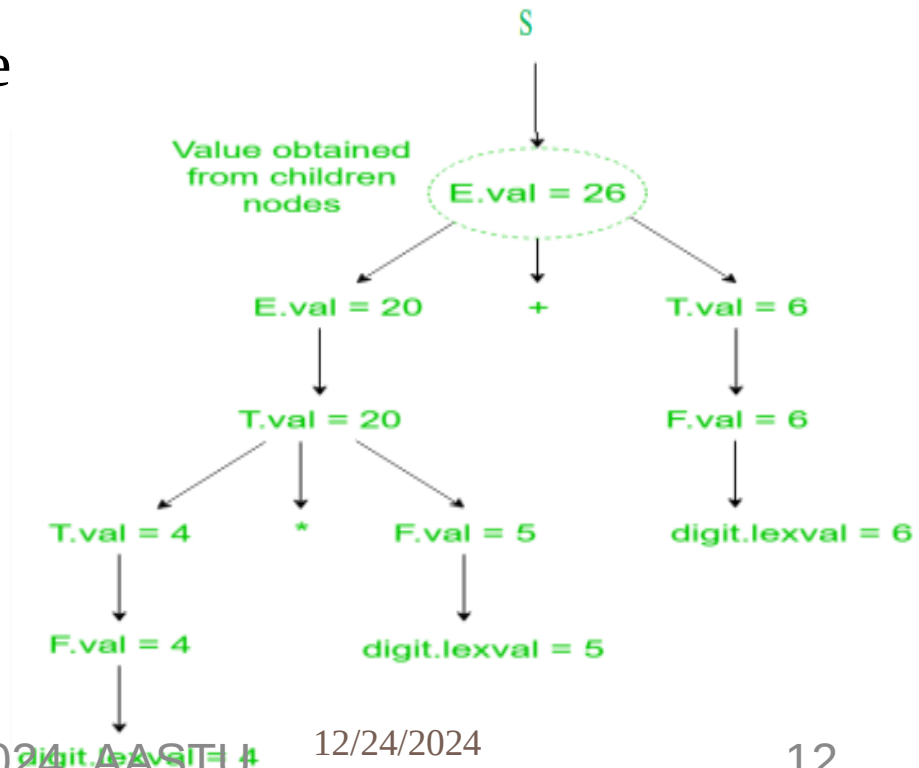
- Step 2: the annotated parse tree is generated and attribute values computed with bottom up manner for the given input string : **4* 5 +6**

1. start from left most bottom up node
2. add data left completely then right
3. value obtain from lexical analyzer.

PRODUCTIONS

SEMANTIC RULES

$S \rightarrow E$	$\text{Print}(E.\text{val})$
$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$
$E \rightarrow T$	$E.\text{val} = T.\text{val}$
$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$
$T \rightarrow F$	$T.\text{val} = F.\text{val}$
$F \rightarrow \text{digit}$	$F.\text{val} = \text{digit}.\text{lexval}$



SDD cont..

9

- S – attribute exercise
- Step 1: construct SDD using semantic rule

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E \text{ n}$	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

SDD cont..

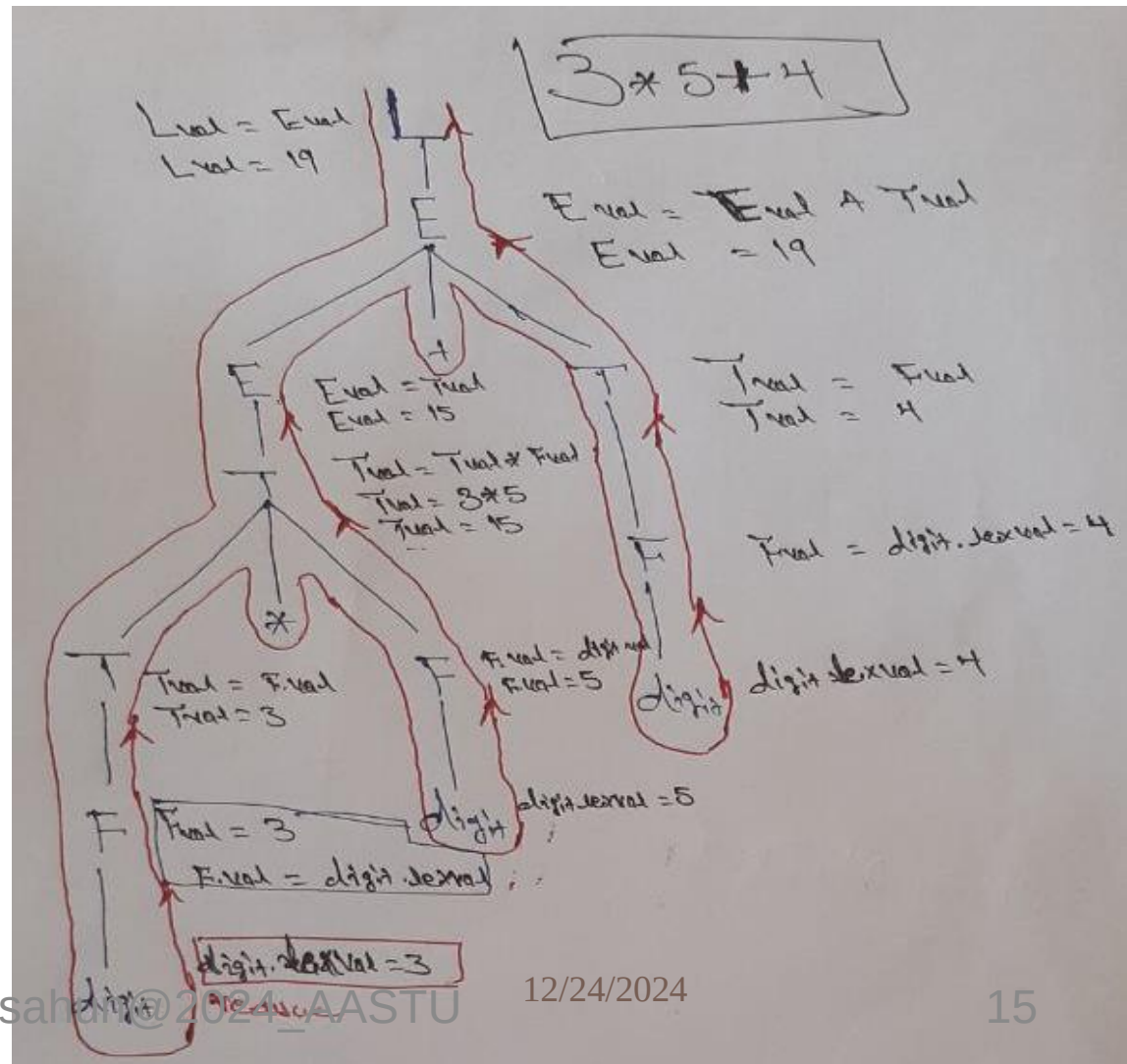
9

- SDD for desk calculator/SDD for evaluation of expressions
- Evaluate the expression $3*5+4n$ using the above SDD both in bottom up and top down approach
- Solution: Bottom up evaluation for this expression is shown below
- In both case first we need to draw the parse tree.
- Then traverse from top to down left to right
- In bottom up approach, whenever there is a reduction ,go to the production and carry out the action.

SDD cont..

9

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E n$	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



SDT cont..

9

- The Principle of Syntax Directed Translation states that the meaning of an input sentence is related to its syntactic structure, i.e., to its Parse-Tree.
- In syntax directed translation, along with the grammar we associate some informal notations and these notations are called as semantic rules.

Grammar + semantic rule = SDT (syntax directed translation)

- In syntax directed translation, every non-terminal can get one or more than one attribute or sometimes 0 attribute depending on the type of the attribute. The value of these attributes is evaluated by the semantic rules associated with the production rule

SDT cont..

9

- In the semantic rule, attribute is VAL and an attribute may hold anything like a string, a number, a memory location and a complex record.
- In Syntax directed translation, whenever a construct encounters in the programming language then it is translated according to the semantic rules define in that particular programming language.

SDT cont..

9

Production	Semantic Rules
$E \rightarrow E + T$	$E.val := E.val + T.val$
$E \rightarrow T$	$E.val := T.val$
$T \rightarrow T * F$	$T.val := T.val * F.val$
$T \rightarrow F$	$T.val := F.val$
$F \rightarrow (F)$	$F.val := F.val$
$F \rightarrow num$	$F.val := num.lexval$

Syntax directed translation scheme

9

- The Syntax directed translation scheme is a grammar with semantic meaning.
- The syntax directed translation scheme is used to evaluate the order of semantic rules.
- In translation scheme, the semantic rules are embedded within the right side of the productions.
- The position at which an action is to be executed is shown by enclosed between braces. It is written within the right side of the production.

Syntax directed translation scheme

9

□ Example

Production	Semantic Rules
$S \rightarrow E \$$	{ printE.VAL }
$E \rightarrow E + E$	{ E.VAL := E.VAL + E.VAL }
$E \rightarrow E * E$	{ E.VAL := E.VAL * E.VAL }
$E \rightarrow (E)$	{ E.VAL := E.VAL }
$E \rightarrow I$	{ E.VAL := I.VAL }

Syntax directed translation scheme

9

- Syntax directed Translation(SDT): Grammar + Semantic rule (Informal Notation)
- Along with the grammar, we are giving semantic actions also such that along with the parsing tasks some other tasks can also be done in parallel like code generation , Intermediate code generation , expression evaluation ,etc.

Syntax directed translation

9

- Syntax Directed Translation has augmented rules to the grammar that facilitate semantic analysis.
- SDT involves passing information bottom-up and/or top-down to the parse tree in form of attributes attached to the nodes.
- Syntax-directed translation rules use
 - 1) lexical values of nodes,
 - 2) constants &
 - 3) attributes associated with the non-terminals in their definitions.

Syntax directed translation

9

- The general approach to Syntax-Directed Translation is to construct a parse tree or syntax tree and compute the values of attributes at the nodes of the tree by visiting them in some order.
- In many cases, translation can be done during parsing without building an explicit tree.

TYPES OF SDT'S

- 1. S –attributed definition
- 2. L –attributed definition

Type's of SDT

9

S-attributed Definition

S stands for synthesized. If an SDT uses only synthesized attributes, it is called as S-attributed SDT

S-attributed SDTs are evaluated in bottom-up parsing, as the values of the parent nodes depend upon the values of the child nodes.

Semantic actions are placed in rightmost place of RHS.

EXAMPLE: $A \rightarrow BC \{ A.a = B.a, C.a \}$

Type's of SDT

9

L-attributed SDT

L-attributed SDT : If an SDT uses both synthesized attributes and inherited attributes with a restriction that inherited attribute can inherit values from left siblings only, it is called as L-attributed SDT.

Attributes in L-attributed SDTs are evaluated by depth-first and left-to-right parsing manner.

Semantic actions are placed anywhere in RHS.

Type's of SDT

9

L-attributed SDT

For example,

```
A -> XYZ {Y.S = A.S, Y.S = X.S, Y.S = Z.S}
```

is not an L-attributed grammar since $Y.S = A.S$ and $Y.S = X.S$ are allowed but $Y.S = Z.S$ violates the L-attributed SDT definition as attributed is inheriting the value from its right sibling.

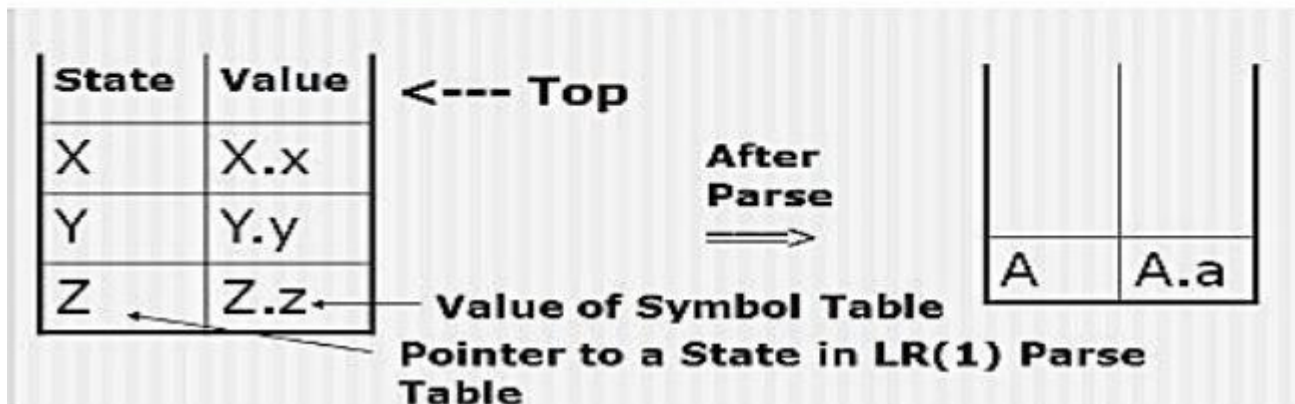
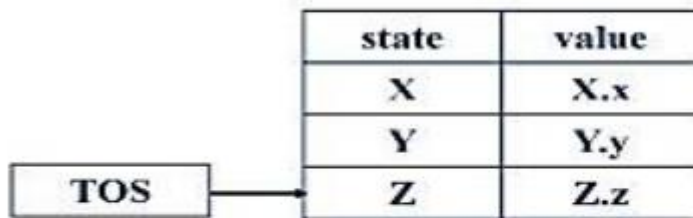
Note – If a definition is S-attributed, then it is also L-attributed but NOT vice-versa. Why??

SDT

9

Bottom up evaluation of S-attributed definition

- e.g. $A \Rightarrow XYZ$ and $A.a := f(X.x, Y.y, Z.z)$



SDT

9

- Syntax-directed definitions with only synthesized attributes(S- attribute) can be evaluated by a bottom up parser (BUP) as input is parsed
- In this approach, the parser will keep the values of synthesized attributes associated with the grammar symbol on its stack.
- The stack is implemented as a pair of state and value.
- When a reduction is made ,the values of the synthesized attributes are computed from the attribute appearing on the stack for the grammar symbols
- implementation is by using an LR parser (e.g. YACC)

SDT

9

- Bottom up evaluation of S-attributed definition
- Example $3*5+4n$

INPUT	state	val	PRODUCTION USED
3*5+4n	-	-	
*5+4n	3	3	
*5+4n	F	3	$F \rightarrow \text{digit}$
*5+4n	T	3	$T \rightarrow F$
5+4n	T *	3 -	
+4n	T * 5	3 - 5	
+4n	T * F	3 - 5	$F \rightarrow \text{digit}$
+4n	T	15	$T \rightarrow T * F$
+4n	E	15	$E \rightarrow T$
4n	E +	15 -	
n	E + 4	15 - 4	
n	E + F	15 - 4	$F \rightarrow \text{digit}$
n	E + T	15 - 4	$T \rightarrow F$
n	E	19	$E \rightarrow E + T$
	E n	19 -	
	L	19	$L \rightarrow E n$

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E n$	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

SDT

9

- Evaluate the expression $4 - 6 / 2$ 2.
- Evaluate the expression $2 + (3 * 4)$
- Bottom up evaluation of S-attributed definition $3 + 4 * 5$
- Find L attributed of below CFG

$T \rightarrow PL$

$P \rightarrow \text{int/float/double}$

$L \rightarrow li, id$

$L \rightarrow id$

input: double id1, id2, id3

Kassam@2024_2_ASTU

12/24/2024

Exercise

9

Find the S attributed of below CFG

$S \rightarrow E$

$E \rightarrow E + E$

$E \rightarrow T$

$E \rightarrow E * T$

$T \rightarrow T * F$

$F \rightarrow \text{digit}$

input: $5 * 10 + 7 + 8 * 20$ Kassahun@2024_AASTU 12/24/2024

Thank You